Complete Project Source Code Navigation Guide & Architectural Map

Project Title: Real-Time Road Pothole Detection and Driver Warning System
Document: Source Code Directory Guide (Handbook Part 8)
Target Audience: Students, academic reviewers, examiners, and software auditors
Project Root Directory: `E:\React\Real\Code\MERN-Projects\final_year`

---

1. Master Source Code Inventory & Quick Navigation Table

Relative Path	Primary Purpose	Key Classes / Functions
run_all.bat	Dual-server automated startup batch script	Windows shell commands (`start cm ...`)
package.json	Node.js project manifest & build scripts	`"scripts"`, `"dependencies"`, `"devDepend`
vite.config.js	Vite bundler build configuration	`defineConfig({ plugins: [react()] })`
tailwind.config.js	Tailwind CSS styling configuration	`module.exports = { content: [...] }`
requirements.txt	Python dependencies specification	Pinned package versions (`ultralyti fastapi`)
index.html	Single-page application HTML entry shell	`<div id="root">`, Google Fonts links
src/main.jsx	React client entry point	`createRoot(document.getElementById(`
src/App.jsx	Main dashboard UI layout & static inspector	`checkBackendHealth()`, `handleFileSele runInference()`
src/components/LiveCameraPanel.jsx	Live camera feed, HUD canvas, and telemetry	`startCameraStream()`, `startInferenceL drawHUD()`

src/utils/audioAlert.js	Web Audio API acoustic hazard alert synthesizer	`AudioAlertManager`, `playHazardBeep()`, `setCooldown()`
pothole-research/scripts/server.py	FastAPI inference REST microservice (Port 5000)	`get_model()`, `detect_image()`, `health_` `run()`
pothole-research/scripts/train.py	YOLOv8 transfer learning training orchestrator	`train_model()`, `detect_device()`, `main`
pothole-research/scripts/validate.py	Model validation & performance logger	`evaluate_split()`, `analyze_fp_fn_samp` `run_evaluation()`
pothole-research/scripts/dashcam.py	Video file / webcam multi-object tracking	`process_dashcam()`, `main()`
pothole-research/scripts/clean_and_prepare_dataset.py	Dataset validation, deduplication, 70:20:10 split	`compute_file_hash()`, `sanitize_yolo_l` `split_dataset()`
pothole-research/scripts/validate_dataset_integrity.py	Zero cross-split data leakage verification	`audit_dataset_integrity()`, `verify_split_leakage()`
pothole-research/scripts/download_dataset.py	Public dataset downloader & VOC XML converter	`download_kaggle_dataset()`, `convert_voc_xml_to_yolo()`
pothole-research/scripts/detect.py	Standalone CLI inference on images/folders	`run_detection()`, `main()`
pothole-research/scripts/generate_evaluation_pr_curve.py	High-resolution 300 DPI PR curve & evaluation generator	`main()`
pothole-research/scripts/update_pr_chart.py	Renders clean title-free & short-titled PR charts	`main()`

pothole-research/data.yaml	Active YOLO dataset configuration file	`path: dataset, train, val, test, nc: 1`
pothole-research/data_multihazard.yaml	Configured multi-hazard dataset specification	`nc: 3, names: {0: pothole, 1: crack, damaged_surface}`
pothole-research/runs/detect/pothole_training/weights/best.pt	Trained neural network weights checkpoint (6.5 MB)	PyTorch serialized state dict (73 lay 3.13M params)
pothole-research/runs/detect/pothole_training/args.yaml	Exact training hyperparameters and augmentations	`epochs: 10, batch: 16, imgsz: 640, de cpu`
pothole-research/runs/detect/pothole_training/results.csv	Epoch-by-epoch loss and validation metrics log	10 epochs of `train/box_loss, val/bo mAP50`, etc.
pothole-research/results/image_inference_logs.csv	Runtime inference telemetry audit log (312 records)	`Timestamp, Filename, Potholes_Detec Latency, Alert`

---

2. Duplicate Filenames Analysis & Resolution

A thorough audit reveals that **18 Python scripts exist in duplicate locations**:

- `final_year/scripts/` (Root scripts folder)
- `final_year/pothole-research/scripts/` (Research subsystem scripts folder)

Exact SHA-256 Integrity Verification:

Every single file with a matching name across `final_year/scripts/` and `final_year/pothole-research/scripts/` has an **identical SHA-256 cryptographic checksum**:

- `check_dataset.py` : Identical
- `clean_and_prepare_dataset.py` : Identical
- `create_condition_comparison.py` : Identical
- `dashcam.py` : Identical
- `detect.py` : Identical
- `download_dataset.py` : Identical
- `evaluate_conditions.py` : Identical
- `generate_phase3_report.py` : Identical
- `low_light_preprocessing.py` : Identical
- `phase2_report.py` : Identical

- `plot_phase3_results.py` : Identical
- `record_hardware_info.py` : Identical
- `server.py` : Identical
- `test_predictions.py` : Identical
- `train.py` : Identical
- `validate.py` : Identical
- `validate_dataset_integrity.py` : Identical
- `visualize_dataset.py` : Identical

Which copy is actually executed by the application?

1. In `run_all.bat` (Line 7):

```
start "Backend - FastAPI YOLOv8n (Port 5000)" cmd /k "python pothole-
research/scripts/server.py --port 5000"
```

2. In `package.json` (Line 8):

```
"server": "python pothole-research/scripts/server.py --port 5000"
```

3. In `README.md` (Line 127):

```
python pothole-research/scripts/server.py --port 5000
```

Operational Rule: The application's automated execution scripts and startup batch files invoke the scripts located under **`pothole-research/scripts/`**. The root `scripts/` folder serves as a mirrored copy in the project root.

---

3. Deep-Dive Source Code Inspection

3.1 Process Launcher: `run_all.bat`

- **Relative Path:** `run_all.bat`
- **Purpose:** Automates single-click local deployment of the full multi-tier application by launching two separate terminal windows for the backend and frontend and opening the browser.
- **Connections:** Spawns `pothole-research/scripts/server.py` on port 5000 and executes `npm run dev` (Vite) on port 5173.

```
6: echo 1. Starting FastAPI ML Backend on Port 5000...
7: start "Backend - FastAPI YOLOv8n (Port 5000)" cmd /k "python pothole-
research/scripts/server.py --port 5000"
8:
9: echo 2. Starting React Vite Frontend on Port 5173...
10: start "Frontend - React Vite (Port 5173)" cmd /k "npm run dev"
11:
12: echo 3. Opening application in browser...
13: timeout /t 3 /nobreak >nul
14: start http://localhost:5173
```

3.2 Backend Inference Service: `pothole-research/scripts/server.py`

- **Relative Path:** `pothole-research/scripts/server.py`
- **Purpose:** Primary backend microservice implementing FastAPI, CORS middleware, image decoding, YOLO model caching, real-time forward pass, bounding-box rendering, hazard advisory classification, and telemetry CSV logging.
- **Key Functions & Classes:**
  - `get_model(custom_path)`: Resolves candidate checkpoint paths and maintains in-memory singleton `MODEL_CACHE`.
  - `detect_image(req: InferenceRequest)`: Handles `POST /api/detect/image`, decodes Base64 to OpenCV BGR, executes `model.predict()`, renders color-coded bounding boxes, classifies hazard severity, writes to CSV log, and returns `InferenceResponse`.
  - `health_check()`: Handles `GET /api/health`, verifies model availability and loaded class names.
  - `get_sample_list()`: Handles `GET /api/samples`, serves pre-encoded road images for quick demonstration.
- **Key Code Excerpts:**

```
34: CANDIDATE_WEIGHTS = [
35:     Path("runs/detect/pothole_training/weights/best.pt"),
36:     Path("runs/pothole_baseline/weights/best.pt"),
37:     Path("pothole-research/runs/detect/pothole_training/weights/best.pt"),
38:     Path("pothole-research/runs/pothole_baseline/weights/best.pt"),
...
63: def get_model(custom_path: Optional[str] = None):
64:     resolved_path, err = resolve_model_path(custom_path)
...
76:         MODEL_CACHE[path_str] = YOLO(path_str)
...
183: @app.post("/api/detect/image", response_model=InferenceResponse)
184: def detect_image(req: InferenceRequest):
...
194:     np_arr = np.frombuffer(raw_bytes, np.uint8)
195:     img = cv2.imdecode(np_arr, cv2.IMREAD_COLOR)
...
207:     results = model.predict(img, conf=req.conf, verbose=False)
...
227:     if conf_val >= 0.70:
228:         box_color = (30, 30, 220)  # Red for high confidence
229:     elif conf_val >= 0.40:
230:         box_color = (0, 140, 255)  # Orange for medium confidence
231:     else:
232:         box_color = (0, 215, 255)  # Yellow for low confidence
...
321:     with open(csv_path, mode="a", newline="", encoding="utf-8") as f:
322:         writer = csv.writer(f)
...
330:         writer.writerow([timestamp_str, req.filename, req.condition,
req.legal_speed_limit,
```

```
331:                         req.conf, potholes_count, avg_conf, highest_conf,
inference_time_ms, ...])
```

3.3 Frontend Dashboard: `src/App.jsx`

- **Relative Path:** `src/App.jsx`
- **Purpose:** Central React component managing state, view toggling (Side-by-Side vs. Full View), image upload handling, backend health monitoring, confidence slider adjustments, legal speed limit configuration, and rendering the inspection dashboard.
- **Key Functions:**
  - `checkBackendHealth()` : Polling function querying `http://localhost:5000/api/health` every 5 seconds.
  - `handleFileSelect(file)` : Validates image file type (`.jpg` , `.png` , `.webp`) and maximum file size (20 MB).
  - `handleInference()` : Converts user image to Base64 and dispatches asynchronous `POST /api/detect/image` .
- **Connections:** Imports `LiveCameraPanel` from `src/components/LiveCameraPanel.jsx` .

```
29: const API_BASE_URL = 'http://localhost:5000';
...
49:   const checkBackendHealth = async () => {
50:     setBackendStatus((prev) => ({ ...prev, checking: true }));
51:     try {
52:       const res = await fetch(`${API_BASE_URL}/api/health`, { method: 'GET' });
53:       if (res.ok) {
54:         const data = await res.json();
55:         setBackendStatus({ online: true, checking: false, info: data });
...
78:   useEffect(() => {
79:     checkBackendHealth();
80:     const interval = setInterval(checkBackendHealth, 5000);
81:     return () => clearInterval(interval);
82:   }, []);
```

3.4 Live Camera & Dashcam Stream Panel: `src/components/LiveCameraPanel.jsx`

- **Relative Path:** `src/components/LiveCameraPanel.jsx`
- **Purpose:** Manages device camera enumeration via WebRTC, video stream playback, client-side canvas frame extraction at 333 ms intervals, HUD bounding-box vector drawing, live FPS telemetry calculation, and audio warning dispatching.
- **Key Functions:**
  - `startCameraStream()` : Requests video stream from `navigator.mediaDevices.getUserMedia()` .
  - `startInferenceLoop()` : Configures `setInterval` timer (333 ms) to capture canvas frames and send to backend.
  - `drawDetectionsOnCanvas()` : Renders glowing bounding boxes and confidence badges directly onto the overlay canvas.
- **Connections:** Communicates with `src/utils/audioAlert.js` to trigger beeps on hazard detection.

```
99:   if (!navigator.mediaDevices || !navigator.mediaDevices.getUserMedia) {
100:    throw new Error('Camera access API is not supported in this browser environment.');
101:  }
103:  const constraints = {
104:    video: selectedDeviceId ? { deviceId: { exact: selectedDeviceId } } : { facingMode:
'environment' }
105:  };
110:  const stream = await navigator.mediaDevices.getUserMedia(constraints);
...
154:  context.drawImage(video, 0, 0, hiddenCanvas.width, hiddenCanvas.height);
155:  const dataUrl = hiddenCanvas.toDataURL('image/jpeg', 0.85);
...
172:  const res = await fetch(`${API_BASE_URL}/api/detect/image`, {
173:    method: 'POST',
174:    headers: { 'Content-Type': 'application/json' },
175:    body: JSON.stringify({ image: dataUrl, conf: confidenceThreshold, legal_speed_limit:
legalSpeedLimit })
176:  });
...
220:  if (data.potholes_detected > 0) {
221:    audioAlert.playHazardBeep();
222:  }
```

3.5 Acoustic Driver Warning: `src/utils/audioAlert.js`

- **Relative Path:** `src/utils/audioAlert.js`
- **Purpose:** Pure Web Audio API acoustic frequency synthesizer. Generates a two-stage frequency-modulated beep without external audio assets, and enforces an acoustic cooldown timer to prevent driver alarm fatigue.
- **Key Class & Methods:**
  - `AudioAlertManager` : Manages `AudioContext` , volume gain, and cooldown state.
  - `playHazardBeep(force)` : Glides an 880 Hz sine wave to 750 Hz, followed by a 660 Hz triangle wave to 550 Hz.
  - `setCooldown(cooldownMs)` : Configures interval threshold (default 2,500 ms).

```
1: class AudioAlertManager {
2:   constructor(cooldownMs = 2500) {
3:     this.audioCtx = null;
4:     this.cooldownMs = cooldownMs;
5:     this.lastAlertTime = 0;
6:     this.isMuted = false;
7:   }
...
49:       const osc1 = this.audioCtx.createOscillator();
50:       const gain1 = this.audioCtx.createGain();
51:       osc1.type = 'sine';
52:       osc1.frequency.setValueAtTime(880, t);
53:       osc1.frequency.exponentialRampToValueAtTime(750, t + 0.12);
...
66:       const osc2 = this.audioCtx.createOscillator();
```

```
67:       const gain2 = this.audioCtx.createGain();
68:       osc2.type = 'triangle';
69:       osc2.frequency.setValueAtTime(660, t + 0.15);
70:       osc2.frequency.exponentialRampToValueAtTime(550, t + 0.30);
```

3.6 Model Training Script: `pothole-research/scripts/train.py`

- **Relative Path:** `pothole-research/scripts/train.py`
- **Purpose:** Loads the pretrained `yolov8n.pt` backbone, validates the presence of `data.yaml`, checks for existing completed checkpoints, detects available compute hardware (CUDA vs. CPU), enables RAM caching, and launches transfer learning.
- **Key Functions:**
  - `detect_device(preferred_device)`: Checks `torch.cuda.is_available()`. Returns device `0` if CUDA is enabled, or `"cpu"` with logical core count.
  - `train_model()`: Calls `model.train()` with hyperparameters (epochs=10, batch=16, imgsz=640, seed=42).

```
22: def detect_device(preferred_device="auto"):
23:     if preferred_device != "auto":
24:         return preferred_device
25:     if torch.cuda.is_available():
26:         return 0
27:     else:
28:         return "cpu"
...
89:     model = YOLO(model_name)
...
105:    results = model.train(
106:        data=str(data_path.resolve()),
107:        epochs=epochs,
108:        imgsz=imgsz,
109:        batch=batch,
110:        patience=patience,
111:        seed=seed,
112:        device=selected_device,
113:        cache=cache,
114:        project=abs_project,
115:        name=name,
116:        exist_ok=True
117:    )
```

3.7 Offline Dashcam Tracking Script: `pothole-research/scripts/dashcam.py`

- **Relative Path:** `pothole-research/scripts/dashcam.py`
- **Purpose:** Processes video files (`.mp4`) or webcam streams frame-by-frame, runs YOLO tracking (`persist=True`), records pothole trajectory histories, computes real-time FPS, and outputs an annotated MP4 video with a FourCC codec.
- **Key Function:** `process_dashcam(source, weights, conf, skip, output_dir)`

```
62:     model = YOLO(str(weights_path))
64:     cap = cv2.VideoCapture(cap_source)
...
78:     fourcc = cv2.VideoWriter_fourcc(*"mp4v")
79:     out_writer = cv2.VideoWriter(str(out_video_path), fourcc, orig_fps, (width, height))
...
105:    results = model.track(frame, conf=conf, persist=True, verbose=False)
```

3.8 Dataset Preparation & Cleaning Pipeline: `pothole-research/scripts/clean_and_prepare_dataset.py`

- **Relative Path:** `pothole-research/scripts/clean_and_prepare_dataset.py`
- **Purpose:** Ingests raw pothole images and normal road backgrounds, verifies image integrity using PIL, removes exact duplicates using SHA-256 hashing, clamps micro-boundary coordinates, generates a reproducible 70:20:10 stratified split, and exports `dataset_cleaning_report.json`.
- **Key Functions:**
  - `compute_file_hash(filepath)`: Computes SHA-256 checksums to eliminate duplicate images.
  - `validate_image_file(img_path)`: Tests image loading with `Image.open().verify()` and `load()`.
  - `sanitize_yolo_bbox(line, class_names)`: Clamps normalized bounding-box coordinates to $[0.0, 1.0]$.
  - `split_dataset(...)`: Partitions images deterministically using seed 42 into train (70%), val (20%), and test (10%).

```
31: def compute_file_hash(filepath: Path) -> str:
32:     hasher = hashlib.sha256()
33:     with open(filepath, "rb") as f:
34:         for chunk in iter(lambda: f.read(65536), b""):
35:             hasher.update(chunk)
36:     return hasher.hexdigest()
...
61: def sanitize_yolo_bbox(line: str, class_names: List[str], tolerance: float = 0.005):
...
94:     if -tolerance <= x1 < 0:
95:         x1 = 0.0
96:     if 1.0 < x2 <= 1.0 + tolerance:
97:         x2 = 1.0
```

3.9 Dataset Leakage Audit Script: `pothole-research/scripts/validate_dataset_integrity.py`

- **Relative Path:** `pothole-research/scripts/validate_dataset_integrity.py`
- **Purpose:** Performs a cryptographic cross-split audit verifying that no image appears in more than one partition ($\text{Train} \cap \text{Val} = \emptyset$, $\text{Train} \cap \text{Test} = \emptyset$, $\text{Val} \cap \text{Test} = \emptyset$). Outputs `dataset_integrity.json` and `validation_report.txt`.
- **Key Function:** `audit_dataset_integrity(base_dir, data_yaml_path)`

```
108:     train_val_overlap = train_hashes.intersection(val_hashes)
109:     train_test_overlap = train_hashes.intersection(test_hashes)
110:     val_test_overlap = val_hashes.intersection(test_hashes)
112:     leakage_passed = (len(train_val_overlap) == 0 and len(train_test_overlap) == 0 and
len(val_test_overlap) == 0)
```

3.10 Dataset Configuration File: `pothole-research/data.yaml`

- **Relative Path:** `pothole-research/data.yaml`
- **Purpose:** Defines dataset split relative paths and class taxonomy for Ultralytics YOLO.
- **Exact Contents:**

```
1: path: dataset
2: train: images/train
3: val: images/val
4: test: images/test
5: nc: 1
6: names:
7:   0: pothole
```

3.11 Model Hyperparameter Configuration: `pothole-research/runs/detect/pothole_training/args.yaml`

- **Relative Path:** `pothole-research/runs/detect/pothole_training/args.yaml`
- **Purpose:** Auto-generated configuration capturing the exact arguments and augmentation parameters used during baseline model training.
- **Key Parameter Excerpts:**

```
3: model: yolov8n.pt
4: data: data.yaml
5: epochs: 10
8: batch: 16
9: imgsz: 640
13: device: cpu
22: seed: 42
43: iou: 0.7
94: hsv_h: 0.015
95: hsv_s: 0.7
96: hsv_v: 0.4
98: translate: 0.1
99: scale: 0.5
103: fliplr: 0.5
105: mosaic: 1.0
111: erasing: 0.4
```

3.12 Runtime Telemetry Log: `pothole-research/results/image_inference_logs.csv`

- **Relative Path:** `pothole-research/results/image_inference_logs.csv`

- **Purpose:** Audit trail of all executed inferences via `POST /api/detect/image`. Contains 312 historical records.
- **Header & Sample Record:**

```
1:
Timestamp,Filename,Condition,Legal_Speed_Limit_kmh,Confidence_Threshold,Potholes_Detected,Avg_C
2: 2026-10-10 22:02:05,img-104.jpg,Not
specified,60,0.25,1,93.1%,93.1%,3506.92,720,720,POTHOLE DETECTED — SLOW DOWN
3: 2026-10-10 22:02:18,normal_road_001.jpg,Not
specified,50,0.25,0,0.0%,0.0%,1393.78,280,150,ROAD SURFACE CLEAR — NO HAZARDS DETECTED
```

End of Source Code Map.